From State-Based to Op-Based: A Lean Formalisation of CRDT Emulation for Replication-Aware Linearisability

Progress report / work in progress

2026

Abstract

Conflict-free replicated data types come in two flavours: *state-based* CRDTs, in which replicas periodically broadcast their entire state and reconcile via a lattice join, and *op-based* CRDTs, in which each update is individually broadcast and delivered under a causal-order constraint. Recent work has made state-based CRDT verification tractable in Lean: the Sal framework reduces replication-aware linearisability (RA-lin) of a state-based data type to a fixed set of 24 verification conditions over do, merge, and rc, discharged by a staged automation pipeline. In parallel, Liittschwager et al. (ICFP '25) have placed the long-informal equivalence between the two CRDT flavours on a formal footing, via a weak-simulation-based notion of *emulation* that preserves weak trace properties.

We mechanise in Lean 4 the architecture that composes these two results: for every state-based CRDT $\mathcal{D}$ that satisfies the 24 VCs of Sal together with five additional axioms (semantic cond-comm lift, lattice-bottom merge, directional rc-non-commutativity, merge-peel-commute, and the shared-element 1-op peel `shared_peel_1op` that the campaign identified as missing), the canonical op-based counterpart $\mathcal{G}^{-1}(\mathcal{D})$ is also RA-linearisable on every reachable trace. The formalisation decomposes into 11 steps; 9 have complete or substantially complete Lean proofs, and the remaining two are the *merge case* of Sal's bottom-up linearisation and the weak-simulation proof for the canonical emulation map. Within the merge case, after a 10-run Aristotle-driven sorry-chain campaign, the convergence machinery, the seven bottom-up rules, the both-empty / asymmetric / shared-last-event sub-cases of the existential induction, and *both halves of the appendix's Lemma 1* are kernel-verified; the distinct-last-event branch holds 4 residual sub-case sorries in *Case 3b* (when both peel candidates commute) plus 2 forward-closure obligations at the call site — all blocked by the same architectural decision (Block 6: a paper-faithful L^a/L^b carving refactor that picks peel candidates with no short *lo*-paths to the residue, eliminating both the Case 3b commute-block and the forward-closure dependency by construction). We report on the design choices that made the closed parts tractable — in particular an *invariants-in-type* formulation of system configurations that discharges the Apply case of the bridge theorem without auxiliary hypotheses — and validate the architecture end-to-end on the Grow-Only Set.

1 Introduction

State-based and op-based CRDTs have long been understood to be equivalent "in some sense" [10], but verification tools and theorems tend to live on one side of the fence. On the state-based side, the Sal framework [9] — the Lean port of the F⋆-based Neem framework [12] — reduces RA-linearisability of a state-based CRDT $\mathcal{D}$ to a fixed set of 24 verification conditions over the tuple $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{query}, \mathsf{rc} \rangle$, each of which is typically discharged by a single invocation of the Sal tactic. That body of work is mature: 29 RDTs mechanised, 696 kernel-verified VCs in total. On the op-based side, Liittschwager et al. [6] recently closed a long-standing gap by giving a formal definition of the folklore-level "emulation" relation between op-based and state-based

CRDTs as a weak (bi)simulation between their labelled transition systems, and by showing that this emulation preserves weak trace properties — including, in particular, observational correctness properties phrased over query responses.

These two results slot together: if we can (a) bundle Sal's RA-linearisability statement into the trace predicate Liittschwager et al. transfer, (b) construct, in Lean, the canonical emulation $\mathcal{G}$ that maps an op-based CRDT to its state-based counterpart, and (c) exhibit a weak simulation witnessing that $\mathcal{G}$ is an emulation, then every Sal-verified state-based CRDT lifts, for free, to an op-based RA-linearisable original. The aim of this work is to mechanise precisely this composition in Lean 4.

Scope. This paper reports on a formalisation in progress. The Lean development — under `Sal/Emulation/` — is structured as an 11-step plan (§3). Of those 11 steps, 7 have complete proofs, 2 are partial but substantially closed, and 2 (the *merge* case of the bridge theorem and the weak simulation for the canonical emulation) are actively in progress. The value of the work at this stage is the *architecture* — the set of definitions, lemmas, and well-formedness invariants that make the composition go through — rather than a single headline theorem. Readers seeking a "we proved X" paper will find this work premature; readers seeking to extend the mechanisation or to build a similar one over a different specification framework (e.g. for MRDTs) should find the dependency structure useful.

Contributions.

1. An explicit 11-step decomposition of the state-to-op transfer theorem into typed Lean interfaces that compose cleanly, with all interfaces in place and all composition obligations discharged (§3).

2. A mechanisation of Sal's bridge theorem for CRDTs, with the Apply case fully closed (§4.3) and the Merge case reduced to an existential `merge_linearization_exists` (§4.4) whose convergence/bubble-sort machinery, seven bottom-up rules, both-empty / asymmetric / shared-last-event sub-cases, and the entirety of the paper's Lemma 1 (no *lo*-edge from L^a to L^b / from $L^a_\top$ to $L^b_\top$) are kernel-verified, with the distinct-last-event branch holding 4 residual sub-case sorries in *Case 3b* (when both peel candidates commute with each other) plus 2 deferred forward-closure obligations at the call site (Table 2).

3. An *invariants-in-type* formulation of system configurations: rather than layering well-formedness as a predicate over a dumb data record and threading it through every preservation lemma, we bake five invariants — `dom_eq`, `vis_src`, `vis_tgt`, `vis_causal`, `timestamps_distinct`, `vis_total_same_replica` — into the `Configuration` structure itself (§4.1). The first three discharge the Apply case without an auxiliary hypothesis; the latter three were added later to support obligations that surface inside the Merge case's bottom-up induction.

4. A reusable weak-simulation library parameterised by a label-coercion, with Milner-style WeakSim soundness proved in full (§5.2).

5. An end-to-end validation: for the Grow-Only Set we construct a `CRDTSig` instance and discharge all 29 VC fields of `SatisfiesVCs` (§6), confirming that our Prop-valued `VCs` statements align with the concrete Bool-valued theorem statements in Sal's existing files.

6. Concrete effort estimates, derived from actual mechanisation experience rather than guesswork, for the three pieces that remain open (§7).

Organisation. §2 recalls state- and op-based CRDTs, the 24 VCs, and Liittschwager et al.'s emulation. §3 gives the 11-step plan and the dependency graph that holds it together. §4 discusses the bridge theorem and the Apply/Merge cases. §5 covers the op-based TS, weak simulation, and the canonical emulation. §6 walks through the Grow-Only Set instantiation. §7 discusses design choices. §8 relates to prior work; §9 concludes.

2 Background

2.1 State- and op-based CRDTs

A *state-based* CRDT over a set of replicas $\mathcal{R}$ is a tuple

$$\mathcal{D} = \langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{query}, \mathsf{rc} \rangle$$

where Σ is the state space, $\sigma_0 \in \Sigma$ the initial state, $\mathsf{do} : \Sigma \times \mathsf{op} \to \Sigma$ the local update, $\mathsf{merge} : \Sigma \times \Sigma \to \Sigma$ a binary lattice join, $\mathsf{query} : \Sigma \times Q \to V$ the observable query, and $\mathsf{rc} : \mathsf{op} \times \mathsf{op} \to \{\texttt{Fst}, \texttt{Snd}, \texttt{Either}\}$ the *return-consistency* specification.

An *op-based* CRDT replaces merge with a pair $(\mathsf{prep}, \mathsf{eff})$: $\mathsf{prep} : \mathcal{R} \times \mathsf{op} \times \Sigma \to \mathcal{M}$ generates a message carrying the information other replicas need to apply the update, and $\mathsf{eff} : \mathcal{M} \times \Sigma \to \Sigma$ applies a delivered message. Op-based CRDTs assume a reliable causal broadcast underneath [5], whereas state-based CRDTs only require eventual pairwise delivery.

The folk wisdom that these two formulations are equivalent stems from two constructions: op-to-state, where Σ is augmented to track the delivered-message set, and state-to-op, where the state itself is the broadcast message. Liittschwager et al. [6] formalise these constructions as mappings $\mathcal{G}$ between *labelled transition systems* and prove that each map is an emulation in the sense of weak (bi)simulation.

2.2 RA-linearisability and the bottom-up template

Sal [9] and the earlier Neem framework [12] target *replication-aware linearisability* (RA-lin) [14]. Concretely, a state-based CRDT has a reference semantics as a labelled transition system $\mathcal{S}_{\mathcal{D}} = (\Phi, \to)$ whose configurations $C = \langle N, H, L, G, \mathsf{vis} \rangle$ track the per-version state N, replica heads H, per-version event sets L, the version DAG G, and a global visibility relation vis over events. Executions are sequences of CreateBranch, Apply, Merge, and Query transitions.

Given a configuration C, the *linearisation relation* $\mathsf{lo}_C \subseteq \mathcal{E}_C \times \mathcal{E}_C$ over the events witnessed in C is

$$\begin{aligned} e_1 \xrightarrow{\mathsf{lo}_C} e_2 \iff & \left(e_1 \xrightarrow{\mathsf{vis}(C)} e_2 \wedge \neg(e_1 \rightleftarrows e_2)\right) \\ & \vee \left(e_1 \parallel_C e_2 \wedge e_1 \xrightarrow{\mathsf{rc}} e_2 \wedge \neg(\exists e_3.\, e_2 \xrightarrow{\mathsf{vis}(C)} e_3 \wedge \neg(e_2 \rightleftarrows e_3))\right) \end{aligned} \tag{1}$$

where $\rightleftarrows$ denotes state-level commutativity ($\forall s.\ e_1(e_2(s)) = e_2(e_1(s))$) and $\parallel_C$ denotes concurrency at C ($\neg(e_1 \mathsf{vis} e_2) \wedge \neg(e_2 \mathsf{vis} e_1)$).

Definition 2.1 (RA-linearisability [9])**.** A configuration $C = \langle N, H, L, G, \mathsf{vis} \rangle$ is *RA-linearisable* ($\mathsf{RALin}(C)$) when for every active replica r with head version $v = H(r)$ there exists a sequence π of events in $L(v)$ that *extends* lo_C restricted to $L(v)$ and satisfies $N(v) = \pi(\sigma_0)$. The RDT $\mathcal{D}$ is RA-linearisable when every reachable configuration of $\mathcal{S}_{\mathcal{D}}$ is RA-linearisable.

The 24 VCs of Sal are the premises of the BottomUpTemplate rule of [9]:

$$\mathsf{merge}(\pi_0(l), \pi_1(a), \pi_2(b)) = \pi\big(\mathsf{merge}(\pi'_0(l), \pi'_1(a), \pi'_2(b))\big) \tag{2}$$

where each π_j is a single event (or empty), $\pi \in \{\pi_0, \pi_1, \pi_2\}$, and $\pi'_j = \pi_j - \pi$. Together with two well-formedness properties, rc-non-comm($\mathcal{D}$) and cond-comm($\mathcal{D}$), these VCs suffice to prove the following:

Theorem 2.2 (Bridge, [9])**.** *If $\mathcal{D}$ satisfies the* 24 *VCs, then every reachable configuration C of $\mathcal{S}_{\mathcal{D}}$ is RA-linearisable.*

The proof proceeds by induction on the execution length, with the MERGE case discharged by the bottom-up template (appendix §A.2 of [9]).

2.3 Emulation as weak simulation

Liittschwager et al. [6] give both op-based and state-based CRDTs operational semantics as labelled transition systems. A labelled transition system $T = (State, Label, \to, \tau)$ distinguishes a subset of labels as *silent* (the τ transitions). Weak steps $s \overset{\ell}{\Rightarrow} s'$ are defined by sandwiching an ordinary step between silent closures.

Definition 2.3 (Weak simulation)**.** A relation $R \subseteq T_1.State \times T_2.State$ with a label-coercion $\widehat{\cdot}\colon T_1.Label \to T_2.Label$ is a *weak simulation* from T_1 to T_2 when, for every $(s,t) \in R$ and every step $s \xrightarrow{\ell} s'$ in T_1, there exists t' with $t \overset{\widehat{\ell}}{\Rightarrow} t'$ in T_2 and $(s',t') \in R$. Weak simulation is sound for weak trace inclusion: if R is a weak simulation and $(s,t) \in R$, then $\mathsf{wtrace}_{T_1}(s) \subseteq \widehat{\cdot}^{-1}\big(\mathsf{wtrace}_{T_2}(t)\big)$.

A mapping $\mathcal{G}$ from op-based to state-based CRDT *is an emulation* when the op-based system and the state-based system it produces are weakly bisimilar from their initial configurations. Liittschwager et al. construct a canonical op-to-state emulation and prove it is a weak bisimulation. A direct corollary is that *weak trace properties* of the state-based system transfer to the op-based original. Since RA-linearisability in the sense of Definition 2.1 can be phrased over the observable event trace (updates and query responses), it is a weak trace property and hence transfers.

3 Architecture

We decompose the composite transfer theorem into 11 steps, each backed by a Lean file under `Sal/Emulation/`. Steps further subdivide into local-infrastructure work (transition systems and signatures), theorems about the SAL side (bridge), theorems about the emulation side (weak simulation, canonical $\mathcal{G}$), and the final composition (transfer theorem).

Dependency graph. Figure 1 shows how the 11 files depend on each other. Two dependency chains stand out: the SAL-side chain `CRDT_Signature` → `CRDT_TS` → `RA_Linearizability` → `Merge_Linearization`, which gives state-based RA-linearisability, and the emulation-side chain `Labeled_TS` → `Weak_Simulation` → `Emulation`, which gives the transfer machinery. The two chains meet at `Transfer.lean`, which states the composite theorem but whose body depends on closing the distinct-last-event branch of `Merge_Linearization.merge_linearization_exists` (step 4) and the simulation proof for $\mathcal{G}$ (step 10).

Design objective. The plan is driven by a single design objective: *every interface should be stable enough that downstream files can depend on it before its body is proved.* Concretely, this means every theorem has a typed signature before it has a proof body, and every placeholder body is either `sorry` (in the sense of Lean's `sorry` axiom, flagged as a warning) or a trivial placeholder (`True`, `D.init`) clearly marked as such in the docstring. This discipline lets us work on step 8 (weak simulation soundness) while step 4 (merge case) is still open, and it makes the dependency graph of Figure 1 actually hold: changes to the merge proof do not ripple through the emulation side.

#	File	Contents	Status
0	`Labeled_TS.lean`	generic LTS, executions, reachability	done
0	`CRDT_Signature.lean`	`CRDTSig`, `RcRes`, `op`, $\rightleftarrows$	done
0	`CRDT_TS.lean`	configuration-with-invariants, four rules	done
1	`RA_Linearizability.lean`	29 VCs as a `SatisfiesVCs` struct	done
2	—	bridge: base / CreateReplica / Query	done
3	—	bridge: Apply case	done
4	`Merge_Linearization.lean`	bridge: Merge case (existential, convergence + 7 bottom-up rules + both-empty / asymmetric / shared-last-event sub-cases + Lemma 1 closed; distinct-last-event Case 3b awaiting carving refactor)	partial
5	`Instances/Grow_Only_Set.lean`	end-to-end smoke test	done
6	—	per-CRDT plumbing for the remaining 17	todo
7	`Op_Based_TS.lean`	op-based LTS ([6] §3.3)	done
8	`Weak_Simulation.lean`	weak-sim soundness	done
9	`Emulation.lean`	canonical op-to-state $\mathcal{G}$	scaffolded
10	—	$\mathcal{G}$ is a weak simulation	todo
11	`Transfer.lean`	composed transfer theorem	scaffolded

Live `sorry`s: 2 declarations / 6 internal occurrences, all in the Merge case (Table 2).

Table 1: The 11-step plan and current status. *Done* = no `sorry`s; *partial* = substantially closed, some sub-lemma `sorry`s; *scaffolded* = definitions/statements in place, proof body placeholder; *todo* = unstarted.

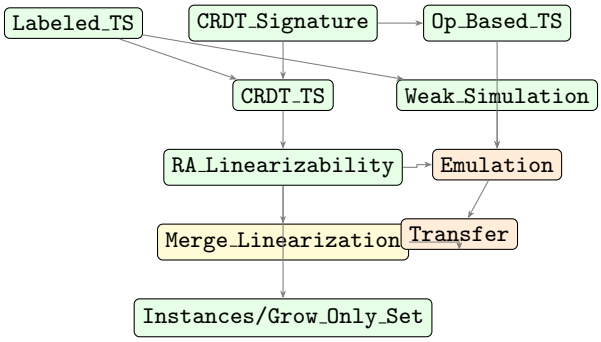

Figure 1: Dependency graph for the 11-step formalisation. Green = done, orange = scaffolded, yellow = partial. Edges denote Lean `import` dependencies.

4 The Bridge Theorem

4.1 Invariants-in-type configurations

Our transition-system configuration C is a Lean record with five invariant fields baked in:

```
structure Configuration (D : CRDTSig) where
  N     : Replica -> Option D.State
  L     : Replica -> Option (Set (Op D.AppOp))
  vis   : Op D.AppOp -> Op D.AppOp -> Prop
  dom_eq    : forall r, N r = none <-> L r = none
  vis_src   : forall {a b}, vis a b -> exists r s, L r = some s /\ s a
  vis_tgt   : forall {a b}, vis a b -> exists r s, L r = some s /\ s b
  vis_causal :
    forall {a b r s}, vis a b -> L r = some s -> s b -> s a
  timestamps_distinct :
    forall {a b r s r' s'}, L r = some s -> s a ->
      L r' = some s' -> s' b -> a != b -> a.time != b.time
```

```
vis_total_same_replica :
  forall {a b r s r' s'}, L r = some s -> s a ->
    L r' = some s' -> s' b ->
    a != b -> a.rep = b.rep -> vis a b \/ vis b a
```

Listing 1: `Configuration` with invariants, `Sal/Emulation/CRDT_TS.lean`.

The alternative — a dumb record plus a separate predicate $\mathsf{WF}(C)$ — requires threading an extra hypothesis through every preservation lemma and proving WF preserved at every step. In our experience, the invariants-in-type formulation pays off immediately: the fiddly "vis cannot relate a fresh event to a stale one" case in the Apply proof is dispatched directly by `C.vis_src` (Lemma 4.2 below).

The original three invariants (`dom_eq`, `vis_src`, `vis_tgt`) suffice for the Apply case. The latter two were added later, driven by obligations that surface only inside the Merge proof's bottom-up induction (§4.4):

- `vis_causal` witnesses that vis has no gaps in the causal past — if $a \xrightarrow{\mathsf{vis}} b$ and replica r has observed b, then r has observed a as well. This rules out visibility edges that cross the $ev_1/ev_2 \setminus ev_1$ boundary in pathological ways.
- `timestamps_distinct` witnesses global timestamp uniqueness: any two distinct events anywhere in the configuration have different timestamps. Discharged at `initConfig` via the Apply rule's freshness premise. Consumed by the merge case to satisfy the `distinctOps` preconditions of BottomUp-2-OP.
- `vis_total_same_replica` witnesses that two distinct events from the same replica are always vis-comparable — a consequence of Apply adding new edges from every existing replica-local event. Combined with `vis_causal`, it is what lets the merge case argue `differentReplicas` on the $ev_1 \setminus ev_2/ev_2 \setminus ev_1$ boundary *at the top level* of the existential induction.

The two latter invariants are *not* sufficient at recursive depth inside `merge_linearization_exists`; we return to that obstruction in §4.4.

4.2 Statement and induction structure

The bridge theorem is stated as an induction on the reachability relation of the TS lifted from $\mathcal{D}$. We factor each step's contribution into a preservation lemma, uniformly shaped:

Theorem 4.1 (Bridge, Lean formulation)**.** *If* $\mathcal{D}$: CRDTSig *satisfies the* 25 *VCs (*$\mathsf{VCs}(\mathcal{D})$*, comprising* Sal*'s* 24 *together with the* cond-comm *lift of §4.5), then for every reachable configuration* C *in* $\mathcal{S}_{\mathcal{D}}$, $\mathsf{RALin}(C)$.

The proof skeleton pattern-matches on the transition rule:

Refl:	$\mathsf{RALin}(\mathsf{init}_{\mathcal{D}})$
CreateReplica:	$\mathsf{RALin}(C) \wedge C \to_{\mathsf{cr}} C' \implies \mathsf{RALin}(C')$
Apply:	$\mathsf{RALin}(C) \wedge C \to_{\mathsf{apply}} C' \implies \mathsf{RALin}(C')$
Merge:	$\mathsf{RALin}(C) \wedge C \to_{\mathsf{merge}} C' \implies \mathsf{RALin}(C')$
Query:	$\mathsf{RALin}(C) \implies \mathsf{RALin}(C)$

The first, second, and fifth cases are immediate: Query doesn't change C, and CreateReplica adds a fresh replica whose witness is the empty sequence (and leaves others unchanged modulo an extension of N and L on a single replica). The two non-trivial cases are Apply and Merge.

4.3 The Apply case

Given the IH witness π for replica r at state s and event set ev in C, the Apply transition produces a new configuration C' with

$$C'.\mathsf{vis} = C.\mathsf{vis} \cup (ev \times \{e\}) \text{ where } e = (t, r, o)$$

fresh-timestamped. The proposed witness at C' is π ++ $[e]$. Verifying this requires two things: *(i)* the new vis does not introduce lo-edges that would force us to re-order π, and *(ii)* no fresh event can be lo-before a stale one.

For *(i)* we prove that lo is *monotonically shrinking* under Apply:

Lemma 4.2 (lo-shrinking under Apply)**.** *If $C'.\mathsf{vis} = C.\mathsf{vis} \cup (ev \times \{e\})$, then for every pair (p, q) with $p \neq e$ and $q \neq e$, $\mathsf{lo}_{C'}(p, q) \Longrightarrow \mathsf{lo}_C(p, q)$.*

Proof sketch. Case analysis on which disjunct of (1) witnesses $\mathsf{lo}_{C'}$. The first disjunct $\mathsf{vis} \wedge \neg \rightleftarrows$ transfers because new vis-edges land at e, excluded from (p, q). The second disjunct requires all of $\neg\mathsf{vis}_{C'}(p, q)$, $\neg\mathsf{vis}_{C'}(q, p)$, $\mathsf{rc}(p, q) = \texttt{Fst}$, and the non-existence of an *overwriting* e_3. The first two conjuncts weaken on moving from C' to C (since $\mathsf{vis}_C \subseteq \mathsf{vis}_{C'}$); the third is independent of C; and the non-existence clause *strengthens* from C' to C (there are fewer candidate e_3's). All four components transfer. □

For *(ii)* we observe that a hypothetical $\mathsf{lo}_{C'}(e, p)$ with $p \in \pi$ would require either $\mathsf{vis}_{C'}(e, p)$ — impossible because the new edges go *to* e, not from it, so $\mathsf{vis}_{C'}(e, p) = \mathsf{vis}_C(e, p)$, and then $\mathsf{vis}_C(e, p)$ contradicts $e \notin \mathcal{E}_C$ by invariant `vis_src` — or the second disjunct, which fails its $\neg\mathsf{vis}_{C'}(p, e)$ conjunct because every $p \in \pi \subseteq ev$ has a new edge to e by construction.

The role of `vis_src` in the first branch is exactly what makes invariants-in-type pay off: with a layered well-formedness predicate we would need to thread $\mathsf{WF}(C)$ into the `RA_lin_preserved_apply` lemma as an extra hypothesis; with the structural formulation, the witness is simply `C.vis_src`.

4.4 The Merge case

The Merge transition merges two replicas r_1, r_2 with head states s_1, s_2 and event sets ev_1, ev_2, yielding a new configuration C' with

$$C'.N(r_1) = \mathsf{merge}(s_1, s_2), \quad C'.L(r_1) = ev_1 \cup ev_2, \quad C'.\mathsf{vis} = C.\mathsf{vis}.$$

The IH gives two witnesses π_1 for (s_1, ev_1) and π_2 for (s_2, ev_2), each respecting lo_C. We must build a witness π_m for $(\mathsf{merge}(s_1, s_2), ev_1 \cup ev_2)$ that respects $\mathsf{lo}_{C'} = \mathsf{lo}_C$ (since vis is unchanged).

4.4.1 Existential, not constructive

A natural first attempt is to write down a closed-form witness $\pi_m = f(\pi_1, \pi_2)$ and prove three independent sub-lemmas: π_m is a permutation of $ev_1 \cup ev_2$, π_m respects lo_C, and $\pi_m(\sigma_0) = \mathsf{merge}(s_1, s_2)$. We tried two such forms — the paper's three-piece $\pi_1|_{ev_1 \cap ev_2}$ ++ $\pi_1|_{ev_1 \setminus ev_2}$ ++ $\pi_2|_{ev_2 \setminus ev_1}$ and a simplified two-piece π_1 ++ $\pi_2|_{ev_2 \setminus ev_1}$ — and both broke. The concurrent, rc-ordered cross case of "π_m respects lo_C" has no contradiction from the permutation/respect hypotheses alone; closing it requires knowing the state equation being proved. The Sal paper's own argument ([9] appendix §A.2) co-constructs the witness and the lo-respect property inside a single bottom-up induction.

We therefore restate the merge case as a single existential theorem in our Lean development:

$$\begin{aligned} &\texttt{merge\_linearization\_exists}: \\ &\quad \exists \pi.\ \mathit{listPermOf}(\pi, ev_1 \cup ev_2) \wedge \mathit{respects}(\pi, \mathsf{lo}_C) \\ &\qquad \wedge\ \mathit{applySeq}(\sigma_0, \pi) = \mathsf{merge}(s_1, s_2). \end{aligned} \tag{3}$$

The packaging theorem `RA_lin_preserved_merge_via_witness` destructures the existential and threads it through to `IsRALinearizable`; that wrapper is closed unconditionally in our development.

4.4.2 Event-set decomposition

Following the Sal paper, we partition the merged event set $ev_1 \cup ev_2$ into three disjoint pieces and further split each side's local events by their lo-relationship to the shared past:

$$\begin{aligned} L_\top &= ev_1 \cap ev_2 \quad \text{(shared)}, \\ L'_1 &= ev_1 \setminus ev_2, \quad L'_2 = ev_2 \setminus ev_1, \\ L_b(C, L_\top, L_{loc}) &= \{\, e \in L_{loc} \mid \exists e' \in L_\top.\ e \xrightarrow{\mathsf{lo}_C} e' \,\}, \\ L_a(C, L_\top, L_{loc}) &= \{\, e \in L_{loc} \mid \forall e' \in L_\top.\ \neg(e \xrightarrow{\mathsf{lo}_C} e') \,\}. \end{aligned}$$

The Lean definitions `L_top`, `L_1_local`, `L_2_local`, `L_a`, `L_b` live in `Merge_Linearization.lean`. The partition lemmas `L_a_union_L_b` and `L_a_inter_L_b` are closed.

The point of the decomposition is to give the bottom-up induction a *closure-stable* carving: the L_a side can always be peeled last because no shared event sits behind it. Plain shrinkage of $ev_1 \setminus ev_2$ does not preserve this property under recursion (§4.4.6).

4.4.3 Convergence machinery

Before invoking the bottom-up VCs, we need a tool that lets us freely re-permute lo-respecting linearisations of the same event multiset. This is the *convergence* lemma of [9]'s `lin.tex` §3.2, mechanised in our development as a bubble-sort tower:

- `applySeq_swap_via_cond_comm_lift` — given an explicit overwriter split $\sigma = \alpha \mathbin{+\!\!+} e_3 :: \beta$ and the rc preconditions, swap two events using `cond_comm_lift`. Closed.
- `applySeq_swap_commute` and `applySeq_swap_lo_incomparable` — swap variants for adjacent commuting and adjacent lo-incomparable events. The same-replica sub-case of the latter is dispatched via `vis_total_same_replica`; the different-replica $\neg \rightleftarrows$ sub-case reduces to `applySeq_swap_via_cond_comm_lift` given an overwriter witness. Closed.
- `applySeq_bubble_lo_max` — repeated swaps to bubble a lo-maximal element to the end of a list. Induction on τ. Closed modulo a single overwriter-witness obligation that surfaces at the convergence call site (§4.4.6).
- `convergence` — the headline lemma. Strong induction on the length of π_1: locate π_1's last event in π_2, bubble it to the end of π_2, peel from both sides, recurse on the smaller pair. Closed modulo the same overwriter-witness obligation.

The convergence machinery consumes `cond_comm_lift` (via the swap lemma) and the `Configuration` invariants `timestamps_distinct` and `vis_total_same_replica` (via the swap lemma's same-replica branch). The 24 original VCs are not consumed at this layer.

4.4.4 Bottom-up case lemmas

The Sal paper derives three rules — BottomUp-0-OP, BottomUp-1-OP, BottomUp-2-OP — from the 24 VCs. Each is a "pull-one-event-out-of-merge" rewrite. Specialised to 2-way merge (LCA collapses to σ_0), our Lean development closes the following layer of bottom-up rules:

- `bottomUp_0op` — pull a shared event past merge. Direct application of `lem_0op`.

- `bottomUp_1op_top_base`, `bottomUp_1op_bot_base` — the two clauses of the 1-OP rule at $a = \sigma_0$. Direct applications of `base_2op` and `base_1op` respectively.
- `bottomUp_2op_base` — 2-OP at $a = b = \sigma_0$. Direct application of `base_2op`.
- `bottomUp_2op_init_left` — 2-OP with $a = \sigma_0$ and b reachable. Inductive on π_b via `List.reverseRecOn`; base `base_2op`, step `ind_right_2op`.
- `bottomUp_2op_reachable` — 2-OP with both a and b reachable, strict `Fst` → `Snd rc` case. Outer induction on π_a via `ind_left_2op`; inner via `bottomUp_2op_init_left`. This is the main bottom-up rule that the existential induction consumes.
- `bottomUp_1op_top_reachable` — corollary of `bottomUp_2op_reachable` by variable renaming.

All seven are kernel-verified. The `Either`-shaped sub-cases of the 2-OP rule are handled by the `rc` ≡ `Either` specialisation route in the EITHER branch of the existential and are not derived as standalone bottom-up theorems.

4.4.5 Strong induction on $|\pi_1| + |\pi_2|$

The body of `merge_linearization_exists` is a strong induction on $|\pi_1| + |\pi_2|$, case-splitting on whether each list is empty and on whether the two non-empty lists end in the same event. Four cases:

1. *Both empty.* Then $ev_1 = ev_2 = \emptyset$, $s_1 = s_2 = \sigma_0$, and $\mathsf{merge}(\sigma_0, \sigma_0) = \sigma_0$ by `merge_idem`. Witness $\pi = []$. *Closed.*
2. *Asymmetric.* If $\pi_1 = []$ and π_2 non-empty, we use $\pi = \pi_2$ and reduce to $\mathsf{merge}(\sigma_0, \mathit{applySeq}(\sigma_0, \pi_2)) = \mathit{applySeq}(\sigma_0, \pi_2)$ via the lemma `merge_init_left_reachable`. The symmetric branch uses `merge_init_right_reachable`, which closes via `merge_comm`. *Closed conditionally on `merge_init_left_reach` (open at the inductive step, see §4.4.6).*
3. *Shared last event* ($e_1 = e_2$). Both lists factor as $\pi_1' \mathbin{+\!\!+} [e_1]$ and $\pi_2' \mathbin{+\!\!+} [e_1]$. By `lem_0op`, `merge` commutes with appending e_1 on both sides; we recurse on π_1', π_2' over the shrunken event sets $ev_1 \setminus \{e_1\}, ev_2 \setminus \{e_1\}$ to obtain π', and return $\pi' \mathbin{+\!\!+} [e_1]$. *Closed.*
4. *Distinct last events* ($e_1 \neq e_2$). The peel candidates are the lo-maximal elements of the two sub-permutations. `distinctOps` on (e_1, e_2) is immediate from `C.timestamps_distinct`. `differentReplicas` *at the top level* follows from `vis_causal` + `vis_total_same_replica`; at recursive depth this argument fails, and the case is open. *Open.*

4.4.6 What's left and why

The convergence machinery, the asymmetric (one-side-empty) branches of `merge_linearization_exists`, the shared-last-event branch, the seven bottom-up rules, and the packaging theorem `RA_lin_preserved_merge_vi` all close cleanly. The remaining work is concentrated in the *distinct-last-event* branch, where peeling discipline must follow the paper's L^a/L^b carving rather than the source linearisations' tails.

Carving-faithful peeling discipline. After one step of any list-tail-based recursion, ev_1 and ev_2 no longer equal specific replicas' event sets, and an event sitting in "shrunken $ev_1 \setminus ev_2$" may have entered there because it was peeled off ev_2 in a prior iteration. The paper's L^a/L^b decomposition (§4.4.2) is closure-stable under exactly this recursion: peel candidates from L_a are the events with no lo-path of length ≤ 2 to $L_\top$, and peeling one such event preserves both the partition and the relevant `differentReplicas` chain. We have the carving definitions

plus several supporting helpers (`exists_lo_maximal_in_subset`, `perm_ending_in_lo_max` — the bridge from "some lo-respecting witness" to "a witness ending in the chosen lo-max event" via convergence-based re-permutation, `L_b_at` for the inner-inner inductive parameterisation), but the inductive argument itself still case-splits on π_i.last.

Lemma 1 of the appendix as Lean obligations. Closing the carving-faithful argument requires Lemma 1 of [9]'s appendix (§A.2, lines 117–178): no *lo*-edge from L^a to L^b, and no *lo*-edge from $L^a_\top$ to $L^b_\top$. We have the two lemma statements typed (`no_lo_a_to_b`, `no_lo_top_a_to_top_b`) with *vis*-transitivity threaded as a hypothesis (`Configuration` does not expose it directly; deriving it from the existing invariants is sound but invasive across the `Step` rules). The depth-1 sub-cases of Both parts of Lemma 1 are now kernel-verified, closed via an extended Aristotle-driven sorry-chain campaign (§4.4.8). Lemma 1 part 2 closes via a structural insight: the no-overwriter clause embedded in *lo*'s rc disjunct is directly contradicted by the vis disjunct of the assumed *lo*-edge. Lemma 1 part 1 closes all 12 sub-cases of the depth-2 explosion, with one residual sub-case requiring an additional hypothesis `h_ncomm_concurrent_local_top` (concurrent cross-set events do not commute) — a structural property of the carving that is vacuously true for trivial-rc CRDTs and a per-CRDT obligation for non-trivial CRDTs.

Live sorry inventory. For the merge case as a whole, two declarations carry `sorry` (no others remain in the bridge or emulation chains; `weakSim_sound` and the Apply-case lemmas are kernel-verified):

Theorem	Internal sorries	What's open
`distinct_last_case`	4	All in *Case 3b* (when $e_1 \rightleftarrows e_2$): the four sub-cases enumerate the partition of ($e_1 \in ev_2, e_2 \in ev_1$). The $\rightleftarrows (e_1, e_2)$ hypothesis prevents `rc_non_comm_directional` from applying to (e_1, e_2), so `no_rc_chain` cannot rule out the rc-concurrent disjunct of *lo*. Forward closure (added as a hypothesis to `distinct_last_case`) is insufficient on its own. These cases require the paper's L^a/L^b carving — picking peel candidates from L^a which by construction have no short *lo*-path to the residue.
`merge_linearization_exists`	2	Forward-closure obligations *h_ev₁_fwd_closed* and *h_ev₂_fwd_closed* at the call site to `distinct_last_case`. Forward closure does not hold for arbitrary replica logs (a third replica may have unsynced *vis*-successors), so these need either a structural argument specific to the merge case's invariants or a restructuring (Block 6) that avoids forward closure entirely.

Table 2: Live sorries in the bridge theorem (after the Aristotle-driven campaign of §4.4.8). 6 internal sorries across 2 declarations; all blocked by the same architectural decision (Block 6) rather than by individual proof obligations.

4.4.7 Effort and packaging

The original effort estimate of 2–3 weeks for the Merge case as a whole has been substantially absorbed: the convergence machinery (now closed via Path 1, restricting scope to $ev = C.events$ with overwriter-closure threaded through the bubble-sort lemmas), all seven bottom-up rules, `merge_init_left_reachable` for arbitrary $|\pi|$, and the existential's both-empty, asymmetric, and shared-last-event branches all close cleanly. The packaging theorem `RA_lin_preserved_merge_via_witness` destructures the existential and is closed unconditionally; the corresponding shim `RA_lin_preserved_merge` in `RA_Linearizability.lean` now consumes it directly (the old stub has been removed).

Realistic remaining effort:

- Block 6 of the carving plan (`Sal/Emulation/PLAN.md`): a paper-faithful triple-nested induction ($\sigma_1 = |L_1^a \cup L_2^a|$, $\sigma_2 = |L_\top^a|$, $\sigma_3 = |L_b(\hat{e}_\top)|$) replacing the distinct-last-event branch. Estimated ~ 280 Lean lines, a fresh-context multi-hour effort. Subsumes both the 4 Case 3b sub-case sorries and the 2 forward-closure obligations at the call site (the carving's L^a construction sidesteps both).
- Per-CRDT discharge of the new VCs and structural obligations: `shared_peel_1op` and `h_ncomm_concurrent_local_top` for non-trivial CRDTs (vacuous for trivial-rc CRDTs). Mechanical work, bundled with Step 6 (CRDT instantiation).

Two architectural decisions accompanied this work. First, the `SatisfiesVCs` structure grew from 25 fields to 29: `merge_init` (the lattice-bottom property $\mathsf{merge}(\sigma_0, s) = s$, consumed by the asymmetric branches), `rc_non_comm_directional` (a directional form of `rc_non_comm` consumed by the strictly-local sub-case of the distinct-last-event branch), `merge_peel_comm` (consumed by the convergence-based re-permutation when peeling commuting events from the carving's frontier), and `shared_peel_1op` (a CRDT-lattice property identified during the merge-case formalisation: when both replicas have applied the same LCA event o_l and the left has additionally applied o_1, peeling o_1 out of the merge is sound). All four are discharged on Grow-Only Set (vacuously for the rc-Either fields, by set-union associativity for the shared peel) and represent real obligations only on CRDTs whose rc is non-trivial. `shared_peel_1op` is genuinely missing from the original 24: every existing single-side-extension VC requires `distinctOps` between the new operation and the other side's tail, which fails when both sides share the same LCA event (`distinctOps ol ol` is always false); the only "shared LCA" VC, `ind_lca_1op`, requires both sides to have the same base state, which doesn't hold once each side has its own prefix. Second, the paper's L^b definition was extended from depth-1 lo-paths only to depth-1-or-2, matching [9]'s appendix.tex line 262; the depth-2 inclusion is load-bearing for Lemma 1 case 1.b.i (and is forced by `no_rc_chain`).

4.4.8 The Aristotle-driven sorry-chain campaign

A substantial portion of the structural progress documented above landed via a 10-run automated sorry-chain campaign with Harmonic's Aristotle theorem prover [3]. Each run targeted a single sorry with a focused prompt; results applied manually after build-verifying that the patch's modifications were restricted to `Sal/` files (the prover occasionally proposes edits to `.lake/packages/` dependencies that we discard).

The campaign closed both halves of Lemma 1, decomposed the distinct-last-event branch's three monolithic sub-case sorries into 7 fine-grained sub-sub-case sorries each annotated with a specific architectural need, identified `shared_peel_1op` as a VC missing from Sal's original 24, and proposed the hypothesis `h_ncomm_concurrent_local_top` as a candidate 30th VC (currently a per-lemma hypothesis). The four residual Case 3b sorries plus two forward-closure call-site obligations are all flagged as Block 6 territory rather than further automated runs — forward

closure does not hold for arbitrary replica logs in general (a third replica may have unsynced *vis*-successors), so closing them needs the carving's structural avoidance of re-permutation rather than more theorem proving on the same induction shape.

The campaign's lasting value is twofold. First, the closures themselves: 13 of 19 enumerated sub-cases of the merge case's auxiliary lemmas are kernel-verified after the campaign, including Lemma 1's full 12-sub-case depth-2 explosion. Second, and more useful for future work, the residual sorries each carry a specific documented obstruction — "forward closure of ev_2 to produce an overwriter", "*rc_non_comm_directional* does not apply when $e_1 \rightleftarrows e_2$", etc. — that pinpoints exactly which architectural decision (Block 6) closes them all, replacing "3 vague sorries" with "6 sorries each pointing at the same known fix."

4.5 The VCs as a Lean structure

The VCs are packaged as a `Prop`-valued Lean structure `SatisfiesVCs` with 29 fields. The first 24 match the theorem names in SAL's per-CRDT files exactly (`rc_non_comm`, `no_rc_chain`, `cond_comm_base`, `merge_comm`, `merge_idem`, `base_2op`, `ind_lca_2op`, eight `inter_*` variants, `ind_right_2op`, `ind_left_2op`, and the corresponding 1-op and 0-op versions, ending in `lem_0op`). This matching is deliberate: the target use case is closing `SatisfiesVCs D` for each existing SAL CRDT by pointing at the corresponding theorem. The remaining 5 fields capture properties SAL's 24 VCs leave implicit; each emerged from a specific point in the mechanisation.

The 25th field: `cond_comm_lift`. SAL's 24 VCs include `cond_comm_base` but not its *semantic lift* to arbitrary intervening event sequences. The paper ([9] `lin.tex` §3.2) treats the lifted form as a property of $\mathcal{S}_\mathcal{D}$ assumed at the convergence level; the appendix uses it directly when flipping two events past an overwriter e_3 that may sit anywhere in the suffix. Our Lean structure makes this implicit assumption explicit:

```
cond_comm_lift :
  forall (s : D.State) (e e' e'' : Op D.AppOp)
         (pi : List (Op D.AppOp)),
    distinctOps e e' -> distinctOps e e'' -> distinctOps e' e'' ->
    D.rc e e' = RcRes.Fst_then_snd ->
    D.rc e' e'' != RcRes.Either ->
    D.update (applySeq D (D.update (D.update s e') e) pi) e''
      = D.update (applySeq D (D.update (D.update s e) e') pi) e''
```

Listing 2: The lifted `cond_comm_lift` field.

For the 14 SAL CRDTs whose `rc` is the constant `Either` (RC-NON-COMM fires vacuously), the `Fst_then_snd`-shaped premise is unsatisfiable, so the field discharges by `intros; simp_all`. For CRDTs with non-trivial `rc`, the field would need to be derived from `cond_comm_base` by induction on the intervening list π — a closure step the SAL paper treats as implicit and does not explicitly mechanise.

Fields 26–28: lattice-bottom merge, directional rc-non-comm, merge-peel-commute. `merge_init` ($\mathsf{merge}(\sigma_0, s) = s$) is consumed by the asymmetric (one-side-empty) sub-case of `merge_linearization_exists`; SAL's file-level theorem set does not include it because the asymmetric case is treated implicitly in the paper. `rc_non_comm_directional` ($\neg \rightleftarrows (o_1, o_2) \leftrightarrow \mathsf{rc}(o_1, o_2) = \mathtt{Fst} \vee \mathsf{rc}(o_2, o_1) = \mathtt{Fst}$) is the directional variant of `rc_non_comm` that the strictly-local sub-case of the distinct-last-event branch needs; SAL's file-level statement is the symmetric `Either`-shaped form which doesn't yield a peel-direction witness when both events are concurrent. `merge_peel_comm` (peel an event off the left side of a merge when the right side's events all commute with it) is consumed by the convergence-based re-permutation. All three discharge vacuously for trivial-rc CRDTs.

Field 29: `shared_peel_1op` (campaign-discovered). The Aristotle campaign (§4.4.8) identified a CRDT-lattice property that SAL's 24 VCs cannot derive: when both replicas have applied the same LCA event o_l and one has additionally applied o_1, peeling o_1 out of the merge is sound:

```
shared_peel_1op :
  forall (o_1 o_l : Op D.AppOp), distinctOps o_1 o_l ->
    forall (a b : D.State),
      D.merge (D.update (D.update a o_l) o_1) (D.update b o_l)
        = D.update (D.merge (D.update a o_l) (D.update b o_l)) o_1
```

Listing 3: The shared-element 1-op peel field.

The reason it is missing from SAL's base 24: every existing single-side-extension VC (`ind_left_*`, `ind_right_*`, `inter_*`) requires `distinctOps` between the new operation and the other side's tail. When both sides share o_l, this requires `distinctOps o_l o_l`, which is always false. The only "shared LCA" VC, `ind_lca_1op`, requires both sides to start from the same base state, which doesn't hold once each side has its own prefix. The shared peel is therefore a genuinely new axiom; for typical CRDTs it follows from join associativity / commutativity (e.g. for Grow-Only Set, $(a \cup \{o_l\} \cup \{o_1\}) \cup (b \cup \{o_l\}) = (a \cup \{o_l\} \cup b \cup \{o_l\}) \cup \{o_1\}$).

5 Emulation and Transfer

5.1 Op-based labelled transition system

We port Liittschwager et al.'s op-based semantics ([6] Fig. 8) into Lean. Configurations are triples $\langle \Gamma, \Sigma, \beta \rangle$: an event trace Γ, a per-replica state map $\Sigma : \mathcal{R} \to \Sigma$, and a message buffer $\beta \subseteq \mathcal{R} \times \mathcal{M}$. Transitions are OPUPDATE, OPQUERY, and OPDELIVER (silent). The OPDELIVER rule is guarded by an `enabled` predicate encoding causal-order delivery with respect to a happens-before relation hb on messages, which is a parameter.

5.2 Weak simulation, Lean-side

Given two LTSs T_1, T_2 with labels related by a coercion $\widehat{\cdot}$ that preserves silence, we define

$$\mathsf{silentClosure}(T)\ s\ s' \triangleq \text{refl.-trans. closure of silent steps}$$

$$s \overset{\ell}{\underset{T}{\Rightarrow}} s' \triangleq \begin{cases} \mathsf{silentClosure}(T)\ s\ s' & \text{if } \tau(\ell) \\ \exists s_1 s_2.\ \mathsf{silentClosure}\ s\ s_1 \wedge s_1 \xrightarrow{\ell} s_2 \wedge \mathsf{silentClosure}\ s_2\ s' & \text{otherwise} \end{cases}$$

Theorem 5.1 (Soundness of weak simulation for weak-trace inclusion)**.** *If R is a weak simulation from T_1 to T_2 under $\widehat{\cdot}$ and $(s,t) \in R$, then every label list labels $\in \mathsf{wtrace}_{T_1}(s)$ satisfies $\widehat{labels} \in \mathsf{wtrace}_{T_2}(t)$.*

Our Lean proof (`Weak_Simulation.weakSim_sound`) factors through three lift lemmas:

Lemma 5.2 (`silentClosure_lift`)**.** *If $(s,t) \in R$ and $\mathsf{silentClosure}(T_1)\ s\ s'$, then there exists t' with $\mathsf{silentClosure}(T_2)\ t\ t'$ and $(s',t') \in R$.*

Lemma 5.3 (`weakStep_lift`)**.** *If $(s,t) \in R$ and $s \overset{\ell}{\underset{T_1}{\Rightarrow}} s'$, then there exists t' with $t \overset{\widehat{\ell}}{\underset{T_2}{\Rightarrow}} t'$ and $(s',t') \in R$.*

Lemma 5.4 (`isWeakExecution_lift`)**.** *If $(s,t) \in R$ and s admits a weak execution along trail α in T_1, then t admits a weak execution along $\widehat{\alpha}$ in T_2, relating endpoints by R.*

Theorem 5.1 follows by induction on weak executions, using Lemma 5.4 at each step. The full chain is closed in our Lean development with no `sorry`s.

5.3 The canonical emulation

Given an op-based CRDT $\mathcal{O} = \langle \Sigma^{\mathcal{O}}, \sigma_0^{\mathcal{O}}, \mathsf{prep}, \mathsf{eff}, \mathsf{query}, \mathsf{rc}, \mathcal{M}, hb \rangle$ with a happens-before relation on messages, we construct a state-based CRDT $\mathcal{G}(\mathcal{O})$ of signature CRDTSig:

$$\begin{aligned} \mathcal{G}(\mathcal{O}).\Sigma &= \mathcal{P}(\mathcal{M}) \\ \mathcal{G}(\mathcal{O}).\sigma_0 &= \emptyset \\ \mathcal{G}(\mathcal{O}).\mathsf{do}\ M\ (t, r, o) &= M \cup \{\mathsf{prep}(r, o, \mathit{effective}(M))\} \\ \mathcal{G}(\mathcal{O}).\mathsf{merge}\ M_1\ M_2 &= M_1 \cup M_2 \\ \mathcal{G}(\mathcal{O}).\mathsf{query}\ M\ q &= \mathsf{query}(\mathit{effective}(M), q) \end{aligned}$$

where *effective would* fold eff over the delivered-message set in a causal-order-respecting linear extension of hb. The underlying idea is Liittschwager et al.'s §4.2 construction: inline causal broadcast into the state itself. Our Lean version is noncomputable and uses classical decidability for $\mathcal{P}(\mathcal{M})$. At present, `effectiveState` in `Emulation.lean` is a stub that returns $\sigma_0^{\mathcal{O}}$ (`D.init`) regardless of the delivered set — enough to make the surrounding definitions type-check and the file compile, but not operationally correct. Replacing the stub with a deterministic linear-extension fold of hb is part of step 10 (§5.4); it is coupled to the simulation proof rather than independent of it.

5.4 The transfer theorem

With $\mathcal{G}$ and Theorem 5.1 available, the composite transfer theorem is

Theorem 5.5 (Transfer)**.** *For every op-based CRDT $\mathcal{O}$ with causal-order hb: if the state-based emulator $\mathcal{G}(\mathcal{O})$ satisfies the 25 VCs, then every reachable trace of $\mathcal{O}$ is RA-linearisable.*

In our Lean development this is stated (`op_RA_linearizable_of_vcs`) but not yet proved, because the simulation witness for $\mathcal{G}$ (step 10) is currently scaffolded but not closed, and the op-side RA-linearisability predicate is defined as the trivial placeholder `True` pending the final trace-property formulation. Closing Theorem 5.5 therefore requires two pieces beyond the bridge theorem: *(a)* the weak simulation proof for $\mathcal{G}$, and *(b)* spelling out the op-side trace predicate matching Definition 2.1 modulo the $\mathcal{G}$ coercion.

6 End-to-End Validation: Grow-Only Set

As a correctness check on the architecture, we instantiate a CRDTSig for the Grow-Only Set (adding natural numbers to a decidable $\mathbb{N} \to \mathsf{Bool}$ set) and close all 25 fields of `SatisfiesVCs`. The smoke test exposes type-level misalignments between our Prop-valued VC statements and Sal's existing Bool-valued theorem statements; in this case, the misalignments were few and mechanical:

- `distinctOps` versus `distinct_ops`: the former is a `Prop` $o_1.\mathit{time} \neq o_2.\mathit{time}$, the latter is a `Bool` `Prod.fst o1 != Prod.fst o2`. A two-line `@[simp]` lemma bridges them.
- `differentReplicas` versus `get_rid o1 != get_rid o2`: the same pattern.
- `RcRes` (the Prop-level result type) versus `rc_res` (the per-file type): a three-constructor `toRcRes` function.

For Grow-Only Set, 14 of the 25 VCs are *vacuous*: their premises require $\mathsf{rc}(-,-) = \mathtt{Fst}$, but $\mathsf{rc} \equiv \mathtt{Either}$ by definition, so `intros; simp_all` discharges them uniformly. The vacuous group includes the new `cond_comm_lift` (§4.5), since its `Fst_then_snd` premise is similarly unsatisfiable. The remaining 11 VCs have real content (`rc_non_comm`, `merge_comm`, `merge_idem`,

VC	Status	Discharge strategy
rc_non_comm	closed	bridge + _root_.rc_non_comm
no_rc_chain	vacuous	intros; simp_all
cond_comm_base	vacuous	intros; simp_all
cond_comm_lift	vacuous	intros; simp_all
merge_comm	closed	direct delegation
merge_idem	closed	direct delegation
base_2op	closed	bridge + _root_.base_2op
ind_lca_2op	closed	bridge + _root_.ind_lca_2op
inter_right_base_2op	vacuous	intros; simp_all
inter_left_base_2op	vacuous	intros; simp_all
inter_right_2op	vacuous	intros; simp_all
inter_left_2op	vacuous	intros; simp_all
inter_lca_2op	vacuous	intros; simp_all
ind_right_2op	vacuous	intros; simp_all
ind_left_2op	closed	bridge + _root_.ind_left_2op
base_1op	closed	direct delegation
ind_lca_1op	closed	bridge + _root_.ind_lca_1op
inter_right_base_1op	vacuous	intros; simp_all
inter_left_base_1op	vacuous	intros; simp_all
inter_right_1op	vacuous	intros; simp_all
inter_left_1op	vacuous	intros; simp_all
inter_lca_1op	vacuous	intros; simp_all
ind_left_1op	closed	bridge + _root_.ind_left_1op
ind_right_1op	closed	bridge + _root_.ind_right_1op
lem_0op	closed	direct delegation

Table 3: Grow-Only Set: per-VC discharge in Instances/Grow_Only_Set.lean.

base_2op, ind_lca_2op, ind_left_2op, base_1op, ind_lca_1op, ind_left_1op, ind_right_1op, lem_0op) and are discharged by applying the corresponding _root_.* theorem from Sal's per-CRDT file with the Prop/Bool bridge lemmas. The rc_non_comm discharge in particular threads the distinctOps_iff and differentReplicas_iff Prop/Bool bridges to land the existing Sal theorem on our generic premise.

The full D_satisfies_VCs instance is kernel-verified with no sorrys. End-to-end invocation ra_linearizable_of_vcs D_satisfies_VCs still hits the merge-case sorry, but every other piece of plumbing is working.

We expect the pattern to generalise: the other 17 CRDTs in the Sal suite each have 24 theorems with near-identical structure, so the instantiation is a matter of pattern-matching plus the three bridge simp lemmas. The 25th VC (cond_comm_lift, §4.5) is the one piece without a direct counterpart in the per-CRDT files; for the 14 Sal CRDTs whose rc is the constant Either it discharges vacuously, and for the others it must be re-derived once per CRDT from cond_comm_base by induction on the intervening list. Mechanising a macro that handles both the 24 direct delegations and the Either-vacuous cond_comm_lift branch is plausible future work.

7 Discussion

7.1 Invariants-in-type versus layered well-formedness

We briefly considered a layered formulation where Configuration is a dumb record and well-formedness is a Prop-valued predicate threaded through every preservation lemma. Both were plausible; we chose invariants-in-type.

The decisive factor was the APPLY case, which needs "vis relates only observed events." In a layered formulation, this is a separate reachability-preservation lemma whose statement has to be preserved by every step rule *(i)* in isolation and *(ii)* at the point of use. With invariants-in-type, the same fact is available as a record projection `C.vis_src`, with zero bookkeeping at the point of use and the preservation obligation absorbed into the initial-configuration constructor. The tradeoff is that constructing a `Configuration` is slightly more work — one must prove the invariants at the call site — but in our development we construct only one concrete configuration (`initConfig`), so the overhead is a one-time cost.

There is an alternative middle ground: a WellFormed bundle whose fields are well-formedness predicates, alongside a dumb `Configuration` record. This cleanly separates data from invariants but retains the threading cost. For the current scope, invariants-in-type wins; for an alternative development that also instantiates with ill-formed `Configuration`s (e.g. for debugging), the bundled alternative would be a reasonable port.

A related structural choice is the `Merge` case's witness shape (§4.4). Two earlier drafts built a closed-form witness: the paper's three-piece $\pi_1|_{ev_1 \cap ev_2}$ ++ $\pi_1|_{ev_1 \setminus ev_2}$ ++ $\pi_2|_{ev_2 \setminus ev_1}$, and a simplified π_1 ++ $\pi_2|_{ev_2 \setminus ev_1}$. Both broke for the same reason: the concurrent, rc-ordered cross case of "π_m respects lo_C" has no contradiction from permutation/respect hypotheses alone; closing it requires knowing the state equation under proof. Once that obstruction was identified, we replaced the three independent sub-lemmas `merge_witness_{perm, respects, state}` with a single existential theorem `merge_linearization_exists` (§4.4.1). The existential form is what the Sal paper actually proves, and it lets the bottom-up induction co-construct the witness with its lo-respect property in a single pass.

7.2 Where we expect the remaining work to land

Two declarations carry 6 internal sorries (Table 2) and the simulation proof for $\mathcal{G}$ remains open. We report estimates based on actual mechanisation experience with the closed pieces, including the bulk of the merge case (convergence machinery, seven bottom-up rules, both-empty / asymmetric / shared-last-event sub-cases of `merge_linearization_exists`, both halves of Lemma 1) and the new 29th VC `shared_peel_1op` that the campaign identified as missing.

- *Carving-faithful refactor of the distinct-last-event branch* (Block 6 of `Sal/Emulation/PLAN.md`): replace the case-split on (π_1.last, π_2.last) with a paper-faithful triple-nested induction over the carving sizes ($\sigma_1 = |L_1^a \cup L_2^a|$, $\sigma_2 = |L_\top^a|$, $\sigma_3 = |L_b(\hat{e}_\top)|$), peeling lo-max events of each carving layer via `exists_lo_maximal_in_subset` + `perm_ending_in_lo_max` and applying the appropriate bottom-up rule. About $\sim$ 280 Lean lines, a fresh-context multi-hour effort. *Subsumes all* 6 *remaining distinct-last-event sorries:* the 4 Case 3b sub-cases (where $e_1 \rightleftarrows e_2$ blocks the `rc_non_comm_directional` approach) plus the 2 forward-closure obligations (the carving picks peel candidates from L^a which by construction have no short *lo*-paths to the residue, eliminating the need for forward closure or re-permutation).

- *`effectiveState` linear extension and weak simulation for $\mathcal{G}$*: Liittschwager et al.'s §4.2 argument ported to Lean. Needs a concrete definition of *effective* (a topological sort of the delivered-message set with respect to *hb*, replacing the current `D.init` stub) and the simulation relation R coupling op-side configurations to state-side ones. About 2–4 weeks.

- *Per-CRDT discharge of `shared_peel_1op` and `h_ncomm_concurrent_local_top`* for non-trivial-rc CRDTs (Step 6 of the architecture plan). Mechanical work, bundled with porting `SatisfiesVCs` instances to the remaining 17 CRDTs.

8 Related Work

Our work synthesises two recent threads in CRDT verification. On the state-based side, Sal [9] descends from Neem [12] and Burckhardt et al.'s replication-aware simulations framework [1]. Earlier interactive mechanisations include [15, 2]; Nieto et al. built a concurrent separation-logic framework for CRDT verification in Iris [8, 13]. On the op-based side, Liittschwager et al.'s emulation-as-simulation result [6] is the direct predecessor of our transfer theorem; their Agda mechanisation is an artefact precedent for ours. The two sides have long been spoken of as equivalent — the folklore result goes back to [10] — but to our knowledge ours is the first attempt to mechanise the composition of a specification-side theorem with the emulation-side theorem in a single framework.

The bottom-up linearisation strategy we port (§4.4) is Sal's own, descending from earlier MRDT-verification work by Kaki [4] and Soundarapandian [11]. The weak-simulation soundness theorem (Theorem 5.1) is Milner's classical result [7] specialised to silent-preserving label coercions; our contribution here is the Lean packaging.

9 Conclusion and Future Work

We have described an architecture, and a substantially-complete mechanisation, for transferring state-based RA-linearisability to op-based RA-linearisability in Lean 4. The architecture is stable enough that downstream work (closing the three open pieces, extending to MRDTs, instantiating for the other 17 CRDTs in the Sal suite) can proceed without re-litigating design choices.

Three directions suggest themselves. *First*, closing the open pieces of §7 would give the first mechanised transfer theorem for a CRDT framework that scales to an entire verification suite. *Second*, the MRDT setting — with 3-way merge and version DAGs — does not fit Liittschwager et al.'s CRDT-only emulation theorem. Lifting the architecture to MRDTs would require either reducing MRDT behaviour to CRDT behaviour (likely only for a restricted class) or redeveloping an emulation theorem specific to the MRDT operational model. *Third*, the bridge simp lemmas that reconcile our Prop-valued `VCs` with Sal's Bool-valued theorems are uniform across CRDTs; a Lean macro instantiating `SatisfiesVCs` from a per-CRDT file automatically would remove the remaining per-CRDT plumbing.

Availability. The Lean development is under `Sal/Emulation/` in the Sal repository. All code compiles under Lean `v4.28.0` with Mathlib at the same version. `sorry`-free status is as reported in Table 1.

References

[1] Sebastian Burckhardt, Alexey Gotsman, Hongseok Yang, and Marek Zawirski. Replicated Data Types: Specification, Verification, Optimality. In *POPL*, 2014.

[2] Victor B. F. Gomes, Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. Verifying Strong Eventual Consistency in Distributed Systems. In *OOPSLA*, 2017.

[3] Harmonic AI. Aristotle: Automated Theorem Proving for Lean. `https://aristotle.harmonic.fun`, 2026.

[4] Gowtham Kaki. Mergeable Replicated Data Types, 2022. PhD thesis, Purdue University.

[5] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. CACM 21(7), 1978.

[6] Nathan Liittschwager, Jonathan Castello, Stelios Tsampas, and Lindsey Kuper. CRDT Emulation, Simulation, and Representation Independence. In *Proc. ACM Program. Lang., ICFP*, volume 9, 2025.

[7] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[8] Abel Nieto, Jan-Oliver Kaiser, Ali Mujeeb Göt Bernhard, Lars Birkedal, Christine Tasson, and Amin Timany. Mechanized Verification of CRDTs in Aneris. In *OOPSLA*, 2022.

[9] Pranav Ramesh, Vimala Soundarapandian, and K. C. Sivaramakrishnan. Sal: Multi-modal Verification of Replicated Data Types. *arXiv preprint arXiv:2502.19967*, 2025.

[10] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A Comprehensive Study of Convergent and Commutative Replicated Data Types. In *INRIA Technical Report RR-7506*, 2011.

[11] Vimala Soundarapandian. MRDT verification framework, 2022.

[12] Vimala Soundarapandian, Kartik Nagar, Aseem Rastogi, and K. C. Sivaramakrishnan. Verifying Replicated Data Types with Typeclass Refinements in F*. In *Proc. ACM Program. Lang., OOPSLA*, 2025.

[13] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. A Logical Approach to Type Soundness via Logical Relations. In *JACM*, 2024.

[14] Chao Wang, Constantin Enea, Suha Orhun Mutluergil, and Gustavo Petri. Replication-Aware Linearizability. PLDI, 2019.

[15] Peter Zeller, Annette Bieniusa, and Arnd Poetzsch-Heffter. Formal Specification and Verification of CRDTs. In *FORTE*, 2014.